Lab Overview

Lab 04

Graphs and Tries

CS 202: Data Structures, Fall 2026

Lead TA: Anas Allawala

Assignment Overview

Duration: For all intents and purposes, 3 hours
Language: C++
Focus: Graphs, Adjacency Matrices (Upper Triangular), Adjacency Lists (Directed), Depth-First Search (DFS), Expression Trees, Tries

Read Before Starting

- Just please read the manual. I reiterate: PLEASE read the manual. It might feel like a long one and a chore to read, but it contains all help you may need.
- **Skeleton code is provided.** Fill in the function bodies; do not rename files, classes, or members. YOU MUST ONLY WRITE CODE IN THE SRC/CODE FOLDER AND NOT ANYWHERE ELSE.
- Signatures and output format must match exactly.
- Memory leaks or out-of-bounds array access will result in mark deductions.

Contents

1 Marking Distribution **3**

2 Testing **3**

3 Submission Guidelines **4**

4 General Information **5**

5 Task 1: The Adjacency Matrix **5**
- 5.1 Class Declaration 5
- 5.2 Relevant DMA Concepts (Cheatsheet) 6
- 5.3 Methods To Implement 6

6 Task 2: The Adjacency List **8**
- 6.1 Class Declaration 8
- 6.2 Relevant Vector Functions (Cheatsheet) 9
- 6.3 Relevant Unordered Map Functions (Cheatsheet) 10
- 6.4 Methods To Implement 10

7 Task 3: The Trie Data Structure **12**
- 7.1 Class Declaration 12
- 7.2 Methods To Implement 13

8 Fin. **14**

1 Marking Distribution

Task	Part	Description	Marks
Task 1	Matrix	`add_edge()`	5
Task 1	Matrix	`remove_edge()`	5
Task 1	Matrix	`has_edge()`	5
Task 1	Matrix	`degree()`	10
		Matrix Subtotal:	**25**
Task 2	List	`add_edge()`	5
Task 2	List	`remove_edge()`	5
Task 2	List	`has_edge()`	5
Task 2	List	`solve_equation()`	10
		List Subtotal:	**25**
Task 3	Trie	`insertWord()`	15
Task 3	Trie	`searchWord()`	15
Task 3	Trie	`startsWith()`	20
		Trie Subtotal:	**50**
		Total:	**100**

2 Testing

Use the following commands in the VS code terminal to compile and run your test cases from the root of the lab folder.

Compile the test runner:

```
g++ -std=c++17 test_runner.cpp -o test_runner
```

To test Task 1 (Adjacency Matrix):

```
./test_runner task1
```

To test Task 2 (Adjacency List):

```
./test_runner task2
```

To test Task 3 (Tries):

```
./test_runner task3
```

To test all tasks and display final score out of 100:

```
./test_runner all
```

Note: The runner recompiles the relevant source files before each task. A failed test may receive partial or zero marks according to the test-case breakdown. If an incomplete function causes a segmentation fault and prevents later tests in the same task from running, temporarily replace the incomplete body with a safe stub while you work on other functions.

3 Submission Guidelines

Please follow these instructions carefully to submit your lab. Failure to adhere to these guidelines may result in a grading penalty or an ungradeable submission.

- **Packaging:** Compress your entire lab directory into a single `.zip` file.
- **Naming Convention:** Name your zip file using the strict format `Lab4_<RollNumber>.zip`. For example, if your roll number is 28100198, your submission must be named `Lab4_28100198.zip`.
- **Platform and Deadline:** All submissions must be uploaded directly to the LMS portal before the official deadline. Dropbox and email submissions will not be catered to.

4 General Information

Welcome to the Graphs and Tries Laboratory Assessment. In this lab, you will explore how to organize and interlink connected data so that graph traversal and string searching operations are as fast as possible. First, you will implement graph structures in computer memory using Adjacency Matrices and Adjacency Lists. Finally, you will implement prefix tries for highly efficient string and dictionary processing.

---

5 Task 1: The Adjacency Matrix

For our first task, you will implement an undirected graph using an Adjacency Matrix. Because this matrix is for keeping track of undirected graphs, rather than having an $n \times n$ 2D array which would have redundant memory, you will create a dynamically allocated upper-right triangular array. For example, if the number of vertices in the graph is 6, then the matrix will have 5 rows, the first row having 5 columns, then 4, then 3 and so on.

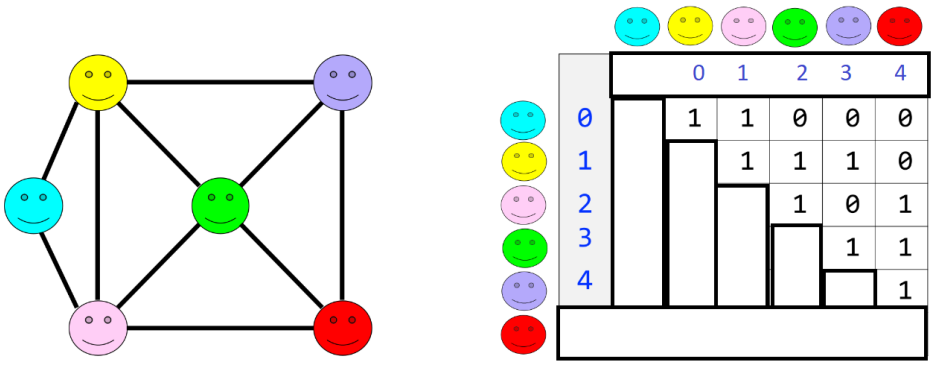


Figure 1: Example of an upper-right triangular adjacency matrix representation.

5.1 Class Declaration

```
1  class Adjacency_Matrix {
2  public:
3      int numVertices;
4      int **matrix;
5
6      Adjacency_Matrix(int n);
7      ~Adjacency_Matrix();
8
9      void add_edge(int start, int end);
10     void remove_edge(int start, int end);
11     bool has_edge(int start, int end);
12     int degree(int node);
13 };
```

Important Note

For this specific task, you must write your implementation in the `Graph.cpp` file.

5.2 Relevant DMA Concepts (Cheatsheet)

Since pointers and Dynamic Memory Allocation (DMA) can sometimes be a nightmare (segmentation fault core dumped yaad hai?), here is a quick refresher to help you set up and clean up your Adjacency Matrix. Just remember, whatever you `new`, you must `delete`!

DMA Operation	Purpose / Behavior
`new Type`	Allocates memory for a single variable of `Type` and returns a pointer to it.
`new Type[n]`	Allocates a contiguous array of `n` elements of `Type` and returns a pointer to the first element.
`delete ptr`	Frees the memory for a single dynamically allocated object pointed to by `ptr`.
`delete[] ptr`	Frees the memory for a dynamically allocated array. (Missing the `[]` causes memory leaks!)
`Type** matrix`	Declares a pointer-to-a-pointer, commonly used for 2D dynamic arrays (like your Adjacency Matrix).
`Allocating 2D Array`	`matrix = new int*[rows]; for(int i = 0; i < rows; i++) matrix[i] = new int[cols];`
`nullptr`	Always set your pointers to `nullptr` after deleting them to avoid dangling pointers.

5.3 Methods To Implement

`Adjacency_Matrix(int n)`

Description: Constructor. Sets `numVertices` as `n`. Allocates the dynamic upper triangular array as described.

`~Adjacency_Matrix()`

Description: Destructor. You are responsible for cleaning up dynamically allocated memory yourself.

`void add_edge(int start, int end)`

Description: Given two arbitrary indices, you are responsible for bounds checking, checking if start and end are the same, and then making the corresponding entry in the matrix into 1 to indicate an edge is present.
Note: Since this is an undirected graph, adding an edge from Node A to Node B is the same as Node B to Node A.

`void remove_edge(int start, int end)`

Description: Given two arbitrary indices, you are responsible for bounds checking, checking if start and end are the same, and then making the corresponding entry in the matrix into 0 to indicate an edge is absent.
Note: Since this is an undirected graph, removing an edge from Node A to Node B is the same as Node B to Node A.

`bool has_edge(int start, int end)`

Description: Given two arbitrary indices, you are responsible for bounds checking, checking if start and end are the same, and then returning true if the corresponding edge is present in the matrix or not.

`int degree(int node)`

Description: Should return the total number of edges a node has. No bounds checking required. "node" will always be a valid index for this function.

6 Task 2: The Adjacency List

For Task 2, you will implement a Directed Graph using an adjacency list. To do this, you will use an `unordered_map` named `list` that maps a `string` to a `vector<string>`.

Here is how the data structure is organized:

- **The Keys (Strings):** Each key in the map represents the name of a specific source node in your graph, such as `"Shahrah e Faisal"`.
- **The Values (Vectors):** The corresponding vector stores the names of all destination nodes that the source node has a directed edge pointing towards.

Example Setup:
If you query `list["Shahrah e Faisal"]`, it might return the vector {`"Malir Expressway"`, `"Mauripur Road"`}. This indicates that there are one-way, directed edges going directly from `Shahrah e Faisal` to both `Malir Expressway` and `Mauripur Road`.

6.1 Class Declaration

```
class Adjacency_List {
public:
    unordered_map<string, vector<string>> list;
    int numVertices = 0;

    void add_edge(string a, string b);
    void remove_edge(string a, string b);
    bool has_edge(string a, string b);
    int solve_equation(string key);
};
```

Important Note

The `Adjacency_List` is used to represent **directed** graphs. For example, `"Sharah e Faisal"` can point to `"Malir Expressway"` but not necessarily the other way around.

6.2 Relevant Vector Functions (Cheatsheet)

Since you might not remember vectors that well (kuch nahi hota, hume bhi CS200 itni nahi yaad thi when we took DS), here's something to help you out in this task. Not all of these are to be necessarily used, it's up to you which functions you want to use for your own implementation (itna tou khudi soch lena chahiye).

Vector Function	Purpose / Behavior
`vec.resize(n)`	Changes the size to `n` elements (n = the new number of elements). New elements are value-initialized, e.g. empty vectors.
`vec.assign(n, val)`	Replaces all contents with `n` copies of `val` (n = how many elements, val = the value each one gets).
`vec.empty()`	Returns **true** if the vector is empty (size == 0), **false** otherwise. $\mathcal{O}(1)$.
`vec.size()`	Returns the number of elements in the vector as `size_t`. $\mathcal{O}(1)$.
`vec.capacity()`	Returns total allocated storage capacity without reallocating. $\mathcal{O}(1)$.
`vec.push_back(val)`	Appends `val` (an already-built element of the vector's type) to the end. Amortized $\mathcal{O}(1)$.
`vec.emplace_back(args)`	Builds a new element directly at the end. `args` are the arguments you would pass to the element's constructor. Amortized $\mathcal{O}(1)$.
`vec.pop_back()`	Removes the last element from the vector. $\mathcal{O}(1)$.
`vec.erase(pos)`	Removes the element at `pos`, where `pos` is an **iterator** (a position like `vec.begin()`), not an index. Shifts remaining elements. $\mathcal{O}(n)$.
`vec.erase(vec.begin()+i)`	Removes the element at **index i** (erase needs an iterator, not an index).
`vec.clear()`	Removes all elements from the vector (size becomes 0). $\mathcal{O}(n)$.
`vec.front() / vec.back()`	Accesses the first or last element directly. $\mathcal{O}(1)$.
`vec.begin() / vec.end()`	Returns iterators to the beginning or past-the-end element.
`vec[i]`	Direct reference access to element at index `i` (no bounds check). $\mathcal{O}(1)$.
`vec.at(i)`	Accesses element at index `i` with bounds checking (throws exception if out of range). $\mathcal{O}(1)$.
`vector<int> copy = vec;`	Makes a full, independent copy (works for any element type, nested vectors too). Later changes to `vec` don't affect it.
`for (auto &x : vec)`	Loops **by reference**: `x` is the real element, so you can modify it or take its address. Without `&`, `x` is a temporary copy.

6.3 Relevant Unordered Map Functions (Cheatsheet)

Since you will be using an `unordered_map<string, vector<string>>` to build your Adjacency List in Task 2, here is a quick guide on how to interact with it. The beauty of an unordered map is that its average time complexity for search, insert, and delete is $\mathcal{O}(1)$!

Map Function	Purpose / Behavior
`map[key]`	Accesses the value mapped to `key`. **Important:** If `key` does not exist, this automatically inserts it with an empty vector! You can safely use `map[key].push_back(val)`.
`map.at(key)`	Accesses the value mapped to `key` with bounds checking. Throws an `out_of_range` exception if the key does not exist.
`map.find(key)`	Searches for `key`. Returns an iterator pointing to the element if found, otherwise returns `map.end()`.
`map.count(key)`	Returns 1 if the `key` exists in the map, and 0 if it does not. (Very useful for quickly checking if a node exists!).
`map.erase(key)`	Removes the key-value pair associated with `key` from the map.
`map.size()`	Returns the total number of key-value pairs currently stored in the map.
`map.empty()`	Returns **true** if the map has no elements (size == 0), **false** otherwise.
`map.clear()`	Removes all elements from the map, destroying them and leaving the size at 0.
`for (auto &pair : map)`	Loops through all elements. Use `pair.first` to access the key (the string) and `pair.second` to access the value (the vector).

6.4 Methods To Implement

`void add_edge(string a, string b)`

Description: Adds string `b` to the vector corresponding to string `a` in the "list" map.

`void remove_edge(string a, string b)`

Description: Removes string `b` from the vector corresponding to string `a` in the "list" map. The string `b` may, or may not exist.

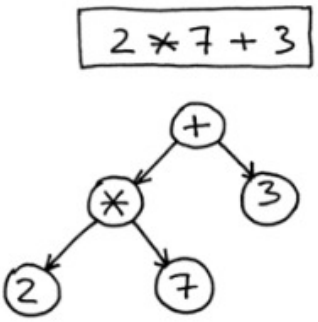
`bool has_edge(string a, string b)`

Description: Return true if string `b` exists in the vector corresponding to string `a` in the "list" map. Returns false if not.

`int solve_equation(string key)`

Description: Have you ever wondered how computers, and specifically calculators, know how to solve mathematical equations? Specifically how do calculators know PEMDAS/-BODMAS or the order of operations and how is it that no matter how convoluted or long an equation is, a simple calculator can solve it? The answer is syntax trees.

Calculators are able to convert mathematical equations into trees and then solve them recursively. For example the equation $2 \times 7 + 3$ will can be converted into a syntax tree like the one below:



Using your previously implemented functions like `add_edge`, the test cases will add edges to your graph and then call `solve_equation` on the root node of your graph.

For example the equation $2 \times 7 + 3$ will be added to the graph in the following manner:

- `graph.add_edge("+", "*")`
- `graph.add_edge("+", "3")`
- `graph.add_edge("*", "2")`
- `graph.add_edge("*", "7")`

The test case will then call `graph.solve_equation("+")`.

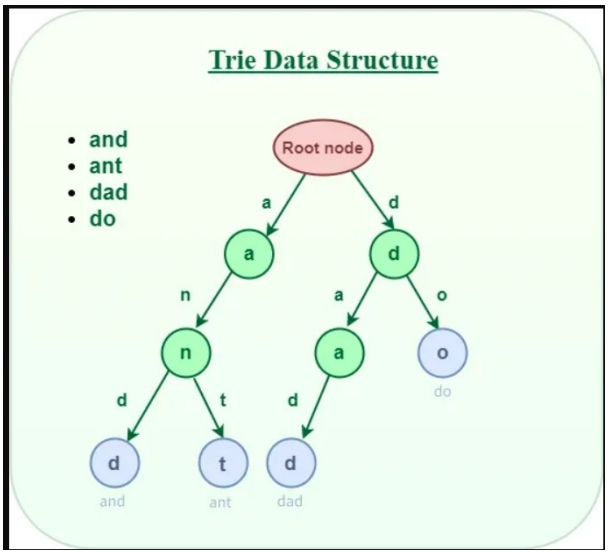
Students are expected to perform a post-order Depth-First-Search traversal to solve the equation and return the correct value.

Note: For simplification, only the 4 main operands: `+`, `-`, `*` and `/` need to be supported. Also, students must use `std::stoi` to convert the `string` representation of integers back to integers.

7 Task 3: The Trie Data Structure

For our final task, you will implement a standard Trie to help efficiently store and search words. You are given a `TrieNode` struct that includes a `bool isEndOfWord` to keep track of whether you have reached the end of a word. Each `TrieNode` contains a dynamically allocated array of 26 `TrieNode` pointers to store possible 26 alphabets that can come after any node. All indices are initialized to `nullptr` in the beginning.

You will also implement a class called `TrieGraph` that has a root node of type `TrieNode`, representing the start of the graph.



7.1 Class Declaration

```
struct TrieNode {
    // pointer array for child nodes of each node
    TrieNode* children[26];

    // Used for indicating ending of string
    bool isEndOfWord;

    TrieNode() {
        // initialize the wordEnd variable with false
        isEndOfWord = false;

        // initialize every index of childNode array with NULL
        for (int i=0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};

class TrieGraph {
public:
    TrieNode *root;

    TrieGraph() {
        root = new TrieNode();
```

```
    }

    void insertWord(string word);
    bool searchWord(string word);
    vector<string> startsWith(string prefix);
};
```

Important Note

For this specific task, you must write your implementation in the `TrieGraph.cpp` file.

7.2 Methods To Implement

`void insertWord(string word)`

Description: Inserts `word` into the Trie.

`bool searchWord(string word)`

Description: Search whether `word` exists in the Trie.

`vector<string> startsWith(string prefix)`

Description: A prefix search that returns, in a vector, all words in the Trie that start with this prefix.

8 Fin.

Congrats on making it to the end of the lab (assuming your scrolling of the manual is directly proportional to the tasks you were able to solve). Unironically, for all intents and purposes, any and all messages within this lab were not meant to be ragebait (though them being perceived as such is something out of my control).

Best of luck!